Fundamentals of API Development

Documenting APIs: Standards & Tools

Day 2 - Session 2

KavinSchool

What We'll Cover

1. Why generated docs beat hand-maintained docs
2. From annotations to a live console: the springdoc pipeline
3. Swagger UI vs Redoc
4. Demo: scoring documentation completeness

Why Generated Docs Win

No separate copy means nothing to forget to update

KavinSchool

The Drawback of Hand-Maintained Docs

A wiki page describing an endpoint updates only when a person remembers to update it. Docs rendered from the OpenAPI file update the moment the file changes, because they have no separate copy to drift from.

> **IMPORTANT**
>
> The spec you wrote in Lab 1.2 is the same document these tools render.

Generated Documentation (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Navy Federal Credit Union — API Exchange Document Library):

- NFCU's developer portal links every API product directly to its specification. When the spec updates, the reference docs update automatically because there is no separate copy to maintain.
- Partners onboarding through the API Exchange read docs rendered from the same OpenAPI file that governs authentication and data access. No wiki page drifts independently.

Generated Documentation (Retail & E-Commerce)

SKIP | REAL-WORLD SCENARIO

Retail & E-Commerce (Shopify — Developer Portal):

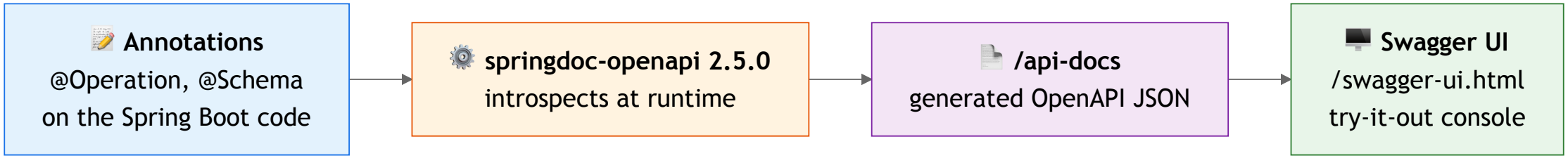
- Shopify's developer portal renders its OpenAPI spec into interactive reference docs. When a new API version releases (2025-01 to 2025-04), the docs regenerate from the new spec.
- No manual doc sync step exists to forget. The spec and the docs are the same artifact, rendered two ways.

From Annotations to a Console

springdoc-openapi does the generating

KavinSchool

The Pipeline



Pin the Right Version

REFERENCE ONLY

This course runs Spring Boot 3.2.5 with `springdoc-openapi` 2.5.0, pinned in the quickstart API's `pom.xml`.

> **WARNING**
>
> The 3.x line of springdoc-openapi only targets Spring Boot 4. Using it against 3.2.5 fails to start. Check pom.xml before trusting a fresh tutorial.

Served at `/api-docs` (spec, a path customized in `application.properties`) and `/swagger-ui.html` (console).

Generating API Docs: The Commands

Step 1 — Add the dependency to `pom.xml`:

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.5.0</version>
</dependency>
```

Step 2 — Start the API. Springdoc auto-generates the spec at runtime:

```
mvn spring-boot:run
```

Step 3 — Open the docs:

What	URL
OpenAPI JSON spec	`http://localhost:8080/api-docs`

Exporting the Generated Spec

To save the generated spec as a file (useful for contract-first workflows or CI):

```
# JSON
curl http://localhost:8080/api-docs -o openapi.json

# YAML (append .yaml to the path)
curl http://localhost:8080/api-docs.yaml -o openapi.yaml
```

NOTE

The spec regenerates every time the app restarts. Add `@Operation`, `@ApiResponse`, and `@Parameter` annotations to your controllers to enrich the generated output with descriptions, examples, and error responses.

Swagger UI vs Redoc: Discussion

You need to publish documentation for two different groups:

1. Internal developers who need to quickly test an endpoint with sample data.
2. External partners who need to read the long-form reference guide.

Which tool would you pick for which group?

> **DISCUSS**
>
> Do you need to write two different specs to serve both?

Swagger UI vs Redoc: The Answer

Tool	Strength	When to use
Swagger UI	Interactive "try it out" console	Exploring and testing during development
Redoc	Clean three-panel reference layout	Public-facing docs, readability first

Both render the same OpenAPI document. Switching renderers never means rewriting the contract.

Accessing Redoc

`springdoc-openapi-starter-webmvc-ui` bundles Swagger UI but not Redoc. Two ways to try it:

CDN (no setup): Paste this URL while the API is running:

```
https://redocly.github.io/redoc/?url=http://localhost:8080/api-docs
```

Static page (one file): Drop `src/main/resources/static/redoc.html` with a `<redoc>` tag loading `/api-docs`, then visit `http://localhost:8080/redoc.html`.

> **ONE SPEC, TWO RENDERERS**
>
> The same `/api-docs` JSON feeds both consoles. No annotation or configuration changes needed to switch.

Documentation Renderers (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Capital One — Internal Developer Portal):

- Capital One provides Swagger UI consoles internally for developers exploring new microservice APIs during development. External-facing partner docs use a Redoc-style reference layout.
- Both renderers consume the same OpenAPI spec from the central microservice catalog. Switching renderers never means rewriting the contract.

Documentation Renderers (Retail & E-Commerce)

SKIP | REAL-WORLD SCENARIO

Retail & E-Commerce (eBay — Developer Program):

- eBay's developer program uses Swagger UI for its sandbox APIs so developers can test calls interactively, and Redoc-styled reference pages for production API documentation so partners can read endpoint contracts.
- One spec, two renderers, zero duplication. The interactive console and the reference guide stay in sync because they share a single source of truth.

Demo: Scoring Documentation Completeness

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoDocCompletenessChecker`.

- Five endpoint doc entries, only some with descriptions and error responses
- Scores each against a completeness checklist
- The `200`-only entries score high on "does it parse" and low on "is it usable"

> Offline, no API key, no network.

Demo: Checking the Live Docs

REFERENCE ONLY

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoLiveDocsCheck`.

- Confirms the quickstart API serves `/api-docs`
- Confirms Swagger UI answers at `/swagger-ui.html`
- Prints the declared `info.version`, so you can compare it against the path prefix

> Start the API first: `mvn spring-boot:run` in `api-dev-setup/quickstart-project`.

Peer Review

A pass over the spec file, not a conversation about vibes

KavinSchool

What a Reviewer Checks

- Every endpoint has a description
- Every schema field has a description and an example
- Every realistic non-2xx response is documented, including error cases

> **CAUTION**
>
> Docs that only show a 200 response are complete and dangerous at the same time.

Peer Review Standards (Financial Services)

REAL-WORLD SCENARIO

Financial Services (UK Open Banking — Conformance Suite):

- UK Open Banking's conformance suite checks every endpoint for descriptions, every schema field for examples, and every non-2xx response for error documentation.
- A spec with only 200 responses fails the review. Financial APIs must document what happens when OAuth consent expires or a token is revoked. The error paths matter as much as the happy path.

Peer Review Standards (Retail & E-Commerce)

SKIP | REAL-WORLD SCENARIO

Retail & E-Commerce (Target — Partner API Review Checklist):

- Target's API partner program review requires every endpoint to document rate-limit (429) and auth-failure (401) responses alongside the happy path.
- Partners integrating inventory and order APIs need to handle errors predictably. Undocumented error codes cause production outages when a fulfillment endpoint returns 503 during peak holiday traffic.

Key Takeaways

1. Documentation generated from the spec cannot drift the way hand-maintained docs do.
2. springdoc-openapi 2.5.0 turns Spring Boot annotations into a live spec and console on this course's stack.
3. Swagger UI and Redoc render the same contract differently; neither changes what the contract says.
4. A useful peer review checks descriptions and error responses, beyond whether the spec parses.

Questions?

Next: Kahoot 2, then a bio break

KavinSchool